

Pega CSA

Scenario-Based Interview Question Bank

310 scenario questions with explanations | aligned to the Pega Platform'25 System Architect mission

Every question in this bank is framed as a real-world scenario — the way interviewers test whether you can apply Pega, not just define terms. Topics and ordering follow the Pega Academy System Architect mission for Pega Platform'25, including the newer areas the platform now emphasizes: Center-out architecture, Constellation UI, Pega GenAI/Autopilot, Pega Blueprint, and Insights reporting. Question style reflects current (2025–2026) Pega CSA interview trends. Explanations are concise talking points — expand on them in your own words and, where you can, tie them to hands-on experience.

Reference: Pega Academy — System Architect mission

(academy.pega.com/mission/system-architect/v8), Pega Platform'25. The CSA exam (PEGACPSA24V1) weights Case Management most heavily (~33%), which is why case-lifecycle, routing, SLA, decisioning, and child/duplicate-case scenarios are covered in depth here.

Contents

1. Low-Code, Center-Out & Microjourney Fundamentals (Q1–Q16)
2. Pega GenAI, Autopilot & Blueprint (Latest Platform) (Q17–Q26)
3. Case Life Cycle Design (Stages, Steps, Processes) (Q27–Q42)
4. Data Model, Data Objects & Relationships (Q43–Q58)
5. Views & Constellation UI Design (Q59–Q74)
6. Routing, Approvals & Cascading Approvals (Q75–Q90)
7. SLAs, Goals, Deadlines & Escalation (Q91–Q106)
8. Automation Shapes & Workflow Automation (Q107–Q122)
9. Decisions: Decision Tables, Trees & When Rules (Q123–Q138)
10. Child Cases, Duplicates, Wait & Dependencies (Q139–Q154)
11. Data Pages, Sourcing & System of Record (Q155–Q170)
12. Data Transforms & Validation (Q171–Q186)
13. Declarative Rules (Q187–Q202)
14. Reuse, Relevant Records & Modular Architecture (Q203–Q218)
15. Security & Access (RBAC, ABAC, Personas) (Q219–Q234)
16. Reporting with Insights (Q235–Q250)
17. Integration (Connectors, Services & REST) (Q251–Q266)
18. DevOps, Testing, Debugging & Troubleshooting (Q267–Q282)
19. Localization, Application Settings & Application Variables (Q283–Q294)
20. Rule Resolution, Class Structure & Core Foundations (Q295–Q310)

1. Low-Code, Center-Out & Microjourney Fundamentals

Q1. A business stakeholder insists the project be delivered by hand-coding in Java for 'full control'. How do you argue for the Pega low-code approach?

Explanation: Explain that Pega's model-driven approach generates and maintains the underlying code from configured rules, giving faster delivery, easier maintenance, upgrade safety, and business-IT collaboration. Custom Java reduces the guardrail compliance score and increases upgrade risk, so it should be a last resort.

Q2. Your client describes their goal as 'let a customer request roadside assistance and get it resolved'. How do you frame this in Pega terms?

Explanation: Frame it as a Microjourney — a customer's journey from an initial need to a resolved outcome, modeled as a Case Type with stages representing the milestones (request, dispatch, service, close).

Q3. The team debates whether to start design from the database schema or the UI. What does Center-out architecture recommend?

Explanation: Center-out starts from the business workflow (the case/microjourney) in the middle, then connects outward to channels (UI/experiences) on one side and systems of record (data/integration) on the other, so business logic stays independent of channel and data specifics.

Q4. A stakeholder asks why they should not just build one giant case type for the whole business. How do you respond?

Explanation: Advise modeling each distinct microjourney as its own case type with a focused outcome; a monolithic case type becomes unmaintainable, hard to reuse, and hard to report on. Smaller, purpose-built case types promote reuse and clarity.

Q5. Business users want to make changes themselves without waiting on IT. Which Pega capability enables this and what are the limits?

Explanation: App Studio lets citizen developers/business architects configure case types, stages, fields, and views visually. Complex rules (activities, advanced integrations) still need Dev Studio, so governance and guardrails apply.

Q6. The client wants the app to work identically on the web and a mobile app. How does the low-code model help?

Explanation: Constellation's responsive, channel-agnostic UI renders the same configured views across web and mobile, so you define the experience once and it adapts, rather than building separate front ends.

Q7. A project has 40 similar service requests across departments. How do you avoid rebuilding the same logic 40 times?

Explanation: Use the Enterprise Class Structure and reuse: build shared behavior in a higher (framework/organization) layer and specialize per department in implementation-layer case types, reusing common data objects, views, and processes.

Q8. Stakeholders can't articulate requirements clearly. Which Pega delivery practice addresses this?

Explanation: Use Directly Capture Objectives (DCO) / Pega Express — collaborative sessions that capture requirements as an executable case design (an MLP first), iterating quickly with working software instead of long specs.

Q9. The sponsor wants value delivered in weeks, not months. What do you propose as the first deliverable?

Explanation: Propose a Minimum Lovable Product (MLP): the smallest slice of the highest-value microjourney that users can actually use, then iterate. This proves value early and reduces risk.

Q10. A developer wants to know where the 'code' for a button lives. How do you explain rules to them?

Explanation: Every element — a view, a field, a process, a decision — is a rule instance stored in the PegaRULES database. At runtime, rule resolution selects the right rule; there is no separate hand-written codebase to maintain for standard behavior.

Q11. The client asks how Pega keeps a 'single view' of the customer across many legacy systems. What do you explain?

Explanation: Pega sits above existing systems; data pages and the data layer federate data from multiple systems of record into one clipboard/case context, so the app presents a unified view without migrating all data into Pega.

Q12. A new team member confuses App Studio and Dev Studio. How do you clarify when to use each?

Explanation: App Studio is the business-friendly, guided authoring environment for case design, data, and UI; Dev Studio exposes all rules and advanced configuration for developers. Most CSA work starts in App Studio, dropping to Dev Studio for specialized rules.

Q13. Business wants to prioritize which microjourney to automate first. What guidance do you give?

Explanation: Prioritize by business value and feasibility — the journey with the highest impact and clearest outcome becomes the MLP. Center-out design lets you deliver it end-to-end before expanding.

Q14. A stakeholder asks what 'Build for Change' means in practice for their app. How do you answer?

Explanation: Because behavior is configured as reusable rules with layered specialization and circumstancing, requirements can change without rewriting the app — you adjust or add rules, keeping the app adaptable over time.

Q15. The client is unsure whether a requirement is a case type or just a data object. How do you decide?

Explanation: If it represents work that progresses through a lifecycle to an outcome, it is a case type; if it represents reference information that supports work (customer, vehicle, address), it is a data object.

Q16. Leadership asks how Pega reduces the 'proliferation of systems' problem. What do you say?

Explanation: By unifying workflow, data, UI, and decisioning on one platform layered above existing systems, Pega removes the need for many disconnected point solutions and gives one consistent place for processes and customer context.

2. Pega GenAI, Autopilot & Blueprint (Latest Platform)

Q17. Your Lead System Architect asks you to quickly stand up a first-draft application from a plain-language description. Which Pega '25 capability do you use?

Explanation: Use Pega Blueprint — a GenAI-powered design tool that generates a suggested application blueprint (case types, stages, steps, data model) from a natural-language description of the business, which you then refine and import into Pega Platform.

Q18. A stakeholder is skeptical that a GenAI-generated Blueprint is production-ready. How do you set expectations?

Explanation: Explain Blueprint accelerates the design/first-draft stage; it is a starting point that the team reviews, adjusts, and hardens with real requirements, data, security, and testing before delivery — not an automatic finished app.

Q19. You want in-context help while configuring rules and generating content in App Studio. Which feature assists you?

Explanation: Pega GenAI features such as Autopilot and in-product GenAI assistance help generate content, suggest configurations, and speed up authoring tasks directly inside the studio.

Q20. Business wants to auto-generate a first cut of the data model and fields for a new case. How can GenAI help and what must you verify?

Explanation: Blueprint/GenAI can propose data objects and fields from the description; you must verify field types, relationships, sources, and security, and align them with the real systems of record before building.

Q21. A reviewer worries GenAI could introduce unreviewed logic. How do you keep governance?

Explanation: GenAI outputs are proposals that enter the normal rule authoring and review workflow — check-in/check-out, testing, and guardrail compliance still apply, so nothing bypasses governance.

Q22. You imported a Blueprint but requirements have since changed. What is the recommended workflow?

Explanation: Iterate the Blueprint or adjust the generated case types in App Studio; Blueprint is meant to be revisited collaboratively, then synchronized/imported as the design evolves.

Q23. A client asks whether GenAI features replace the System Architect role. How do you respond?

Explanation: No — GenAI accelerates repetitive design/authoring tasks, but the architect still validates requirements, ensures best practices and security, integrates real systems, and makes design trade-offs.

Q24. You need to translate a business narrative into stages and steps rapidly during a workshop. How does Blueprint support DCO sessions?

Explanation: Blueprint turns the narrative into a visual, shareable case design in real time, making DCO/Pega Express workshops faster and more collaborative before any building begins.

Q25. Stakeholders want to experiment with several application shapes before committing. How does Blueprint help?

Explanation: Because it generates designs quickly and cheaply from descriptions, teams can explore multiple design options early and pick the best direction before investing in build.

Q26. A security lead asks about data handling when using GenAI features. What is the responsible answer for a CSA?

Explanation: Follow the organization's and Pega's guidance on what data is shared with GenAI services, avoid exposing sensitive data unnecessarily, and confirm configuration against enterprise data-governance policies.

3. Case Life Cycle Design (Stages, Steps, Processes)

Q27. A loan process has phases: Application, Verification, Underwriting, and Closing. How do you model these in the case life cycle?

Explanation: Model each phase as a primary stage arranged left to right; within each stage add steps (processes and tasks) that accomplish that phase's work, so the lifecycle mirrors the business milestones.

Q28. Most applications follow a normal path, but some need a special 'Appeal' path. How do you model the exception?

Explanation: Add an alternate stage for Appeal reached via a change-stage action or condition, keeping the primary stages as the happy path and handling deviations without cluttering it.

Q29. Within a stage you need three sequential user tasks. What is the right construct?

Explanation: Add a process (flow) to the stage containing the sequential steps/assignments, or add individual steps to the stage that run in order.

Q30. Business wants two steps in a stage to happen at the same time. How do you configure that?

Explanation: Set the stage or process to run steps in parallel (parallel processes/steps), so both proceed concurrently rather than one after the other.

Q31. A case must automatically move to the next stage when the current stage's work finishes. How is this handled by default?

Explanation: Stages transition automatically when their steps complete (automatic stage transition). You can also force movement using change-stage actions when business rules require it.

Q32. A manager wants a case to be resolvable from within a stage. How do you resolve a case in the lifecycle?

Explanation: Set the case status to a Resolved-* value (e.g., via a step or when the process reaches its end), which closes the case.

Q33. The client wants the case to show clear progress like 'Verification in progress' to users. How do you achieve that?

Explanation: Set meaningful case statuses per stage/step and use stage names plus user instructions so the portal reflects where the case is in its lifecycle.

Q34. You need one step that runs automatically with no user involvement, and another that waits for a person. How do you differentiate them?

Explanation: Use an automation/utility step (or automation shape) for the automatic work, and an assignment step (Collect information / Approve) for the human task.

Q35. Business decides a whole stage is unnecessary for certain cases. How do you handle it without deleting the stage?

Explanation: Configure the stage to be skipped based on a condition (skip a stage/process), so eligible cases bypass it while others still use it.

Q36. Stakeholders want to reopen a resolved case under certain conditions. How do you design for that?

Explanation: Enable a reopen capability so resolved cases can move back into an appropriate stage, driven by a condition or user action, restarting the relevant processing.

Q37. The lifecycle keeps growing complex. How do you keep it maintainable and reusable?

Explanation: Break reusable processes into their own flows/case types, keep stages focused on business milestones, and reuse shared steps rather than duplicating them across stages.

Q38. A step should only run when the customer selected 'premium' coverage. How do you control step execution?

Explanation: Add a start/stage condition or a 'skip when' condition on the step/process using a when rule, so it runs only for premium coverage cases.

Q39. The business wants users to be able to add optional actions (like 'Update contact info') any time during a case. How?

Explanation: Configure optional actions (case-wide local actions) available in the portal, letting users perform them without advancing the main flow.

Q40. A case must pause until an external event occurs before continuing. How do you model the pause?

Explanation: Insert a Wait step configured to pause until a condition is met or a dependency (e.g., a child case) is satisfied, then resume automatically.

Q41. Two teams disagree on stage order. How do you validate the design quickly with stakeholders?

Explanation: Use the App Studio case designer's visual lifecycle (or a Blueprint) to walk stakeholders through stages and steps interactively and adjust live during the review.

Q42. A step needs to launch a completely separate process defined elsewhere. Which shape/step do you use?

Explanation: Use a subprocess step that calls another process/flow, either waiting for it (call) or running it in parallel (spin-off).

4. Data Model, Data Objects & Relationships

Q43. A case needs customer details, and the same customer structure is reused across several case types. How do you model it?

Explanation: Create a reusable data object (data type/data class) for Customer with its fields, and reference it from each case type rather than redefining the fields each time.

Q44. You must capture one primary address but also a list of previous addresses. Which property modes fit?

Explanation: Use an embedded single page for the primary address and a page list (embedded data object list) for the previous addresses.

Q45. A field should hold multiple selected options like a list of chosen services. Which mode do you use?

Explanation: Use a value list (multiple simple values) or a page list if each selection needs multiple attributes.

Q46. A customer has exactly one vehicle record and you want it embedded in the case. How do you relate them?

Explanation: Create a data relationship (embedded) referencing the Vehicle data object as a single page within the case's data model.

Q47. You need to link a case to a customer that lives in an external CRM, without copying all fields into the case. How?

Explanation: Use a data reference sourced from a data page/data object so the case references the external record on demand instead of storing a full copy.

Q48. The team debates whether Customer should be a case type or a data object. How do you decide?

Explanation: If Customer just supplies information and does not progress through a lifecycle, it is a data object; only make it a case type if 'onboarding a customer' is itself work with an outcome.

Q49. A drop-down of vehicle models must depend on the selected make. How do you structure the data?

Explanation: Source the model list from a data page keyed/parameterized by make, so selecting a make filters the available models (dependent/cascading fields).

Q50. You need to store a set of key-value settings accessed by name. Which property mode fits?

Explanation: Use a value group (keyed simple values) or a page group if each entry needs multiple attributes.

Q51. Stakeholders want a field validated to a fixed set of allowed values. How do you model the field?

Explanation: Define the field with a picklist/local list or source its values from a data object, constraining input to valid options.

Q52. A calculated field (total = price × quantity) must always stay correct. How do you model it?

Explanation: Model it as a calculated/declarative field using a Declare Expression so it recomputes automatically whenever price or quantity changes.

Q53. You need the same 'Address' fields to look and behave identically everywhere. What is the best practice?

Explanation: Define Address once as a data object with its fields (and a reusable view), then embed/reference it everywhere for consistency and single-point maintenance.

Q54. The business wants to report on individual line items inside a case's order list. What must you consider?

Explanation: Embedded list data sits in the case BLOB and is not directly queryable; expose it via a Declare Index (or optimize properties) so line items land in a reportable table.

Q55. A property will be frequently filtered and sorted in reports. What do you do at design time?

Explanation: Expose/optimize the property to a dedicated database column so queries use a real column instead of unpacking the BLOB, improving performance.

Q56. You must model a one-to-many relationship: one order with many items. How do you represent it?

Explanation: Represent items as a page list of an Item data object embedded in the Order case/data object, giving a one-to-many structure.

Q57. The client asks how data objects differ from case data. How do you explain the split?

Explanation: Case data captures the specifics of a single unit of work and its lifecycle; data objects are reusable reference entities (customers, products) that support many cases.

Q58. You need default values populated when a case is created. Where do you set them?

Explanation: Set defaults in the case's data model/data transform (e.g., pyDefault) so the fields initialize with the right values on case creation.

5. Views & Constellation UI Design

Q59. Users need one screen to collect contact details and another read-only summary screen. Which UI construct do you use for each?

Explanation: Use a form/collect-information view for data entry and a read-only/review view for the summary; Constellation views are the reusable UI building blocks for each step.

Q60. The same 'Customer details' fields appear on several steps. How do you avoid rebuilding the layout each time?

Explanation: Create a reusable view for Customer details and embed it wherever needed, so one definition drives the UI across steps.

Q61. A field should appear only when the user selects 'Other' in a dropdown. How do you configure this?

Explanation: Add a visible-when condition on the field driven by a when rule/expression checking the dropdown value, so it shows only when 'Other' is selected.

Q62. The client wants consistent branding — fonts, colors, spacing — across the whole app. Where do you configure this?

Explanation: Configure styling centrally through the Constellation theme/design tokens so branding applies app-wide without editing individual views.

Q63. A list of order items must render as an editable table where users add and remove rows. How?

Explanation: Use a table/list view bound to the item page list with add-row and delete-row actions, mapping each column to an item field.

Q64. Business wants the layout to reflow nicely on phones and tablets. How does Constellation handle responsiveness?

Explanation: Constellation views are responsive by design and adapt to screen size using the design system, so the same view works across devices without separate builds.

Q65. A user action (button) must refresh part of the screen and run a calculation. How do you configure it?

Explanation: Attach an action set to the control's event (click) to run the calculation/data transform and refresh the relevant section, without a full page reload.

Q66. You need to guide users with instructions on a complex step. How do you present guidance?

Explanation: Add step/assignment instructions and helper text in the view, and set clear case status, so users know what to do at each point.

Q67. Different personas should see different fields on the same case. How do you tailor the UI per persona?

Explanation: Use visibility conditions and persona/role-based view configuration so each persona sees only the fields relevant and permitted to them.

Q68. A required field is being submitted empty. How do you enforce entry on the client and server?

Explanation: Mark it required for immediate client-side feedback, and back it with server-side validation so the rule is enforced reliably even if the client is bypassed.

Q69. The client wants a wizard-style multi-screen flow for a single user with Back/Next. How do you build it?

Explanation: Configure a multi-step process presented as sequential screens to one user (screen-flow style), enabling navigation between the screens.

Q70. A currency amount must display formatted with separators and two decimals. How do you handle formatting?

Explanation: Configure the field's type/format settings so the control renders currency formatting consistently, rather than formatting manually.

Q71. Stakeholders ask for a dashboard landing page with widgets. How do you approach it in Constellation?

Explanation: Configure a portal/landing page with the appropriate Constellation widgets (lists, insights, actions) tailored to the persona's needs.

Q72. A section of the form should be collapsible to reduce clutter. How do you achieve this?

Explanation: Use a collapsible/region layout container in the view so users can expand or hide that group of fields.

Q73. The team is tempted to hand-write custom HTML for a view. What do you advise?

Explanation: Prefer configured Constellation views; custom markup reduces maintainability, breaks responsiveness/theming, and complicates upgrades — use it only when unavoidable.

Q74. You need a field's label and behavior to be reused with consistent formatting everywhere. What do you use?

Explanation: Define the field once on the data object with its type/format; reusing the field carries its consistent presentation across views.

6. Routing, Approvals & Cascading Approvals

Q75. An assignment must go to the specific operator who created the case. How do you route it?

Explanation: Route the assignment to a user (worklist) targeting that operator, e.g., the case creator/owner, so it lands on their personal worklist.

Q76. Work should be handled by whichever team member is free, not one named person. How do you route it?

Explanation: Route to a work queue (workbasket) shared by the team, so any qualified member can pull and work the assignment.

Q77. Approvals must go to the submitter's manager, whoever that is. How do you route dynamically?

Explanation: Route based on the reporting structure (to the manager of the operator) using the organization hierarchy, so it resolves the correct manager at runtime.

Q78. A high-value case needs approvals from several levels of management in sequence. Which feature fits?

Explanation: Use cascading approvals, which route the case up the reporting chain through successive approval levels based on a condition (e.g., amount thresholds).

Q79. Approval routing should depend on the requested amount (e.g., >10k needs a director). How do you configure cascading approval levels?

Explanation: Configure the cascading approval to use an authority matrix/decision (e.g., a decision table) that determines how many levels or which roles must approve based on amount.

Q80. Only specialists with a certain skill should get certain assignments. How do you ensure the right people receive them?

Explanation: Use skill-based routing to a work queue filtered by operator skills, so only appropriately skilled users receive those assignments.

Q81. A reviewer can approve or reject; each outcome should take a different path. How do you model this at the assignment?

Explanation: Provide approve/reject actions at the approval step; connectors from the outcome route the case down the corresponding path.

Q82. An operator is on leave and their worklist is piling up. How does a manager redistribute the work?

Explanation: Use the manager portal's reassign/transfer capability to move assignments from that operator's worklist to another operator or a work queue.

Q83. Business wants approvals to consider both amount and region. How do you drive the routing decision?

Explanation: Base the routing/approval decision on a decision table evaluating amount and region together to pick the approver(s).

Q84. A single approval sometimes needs one approver and sometimes a committee. How do you handle both?

Explanation: Use conditional routing: a decision determines whether to route to a single approver or spin off parallel approvals to a committee (multiple approvers) as needed.

Q85. You need round-robin distribution of work across a team. How do you approach it?

Explanation: Route to a work queue and use a get-next-work/assignment distribution strategy (or load-balancing logic) so work is spread across available operators.

Q86. A case should escalate to a supervisor if not approved in time. How do you combine routing with time limits?

Explanation: Attach an SLA to the approval assignment whose escalation action reassigns/notifies the supervisor when the deadline passes.

Q87. Approvers keep approving their own submitted cases, which violates policy. How do you prevent self-approval?

Explanation: Route the approval away from the submitter (e.g., to their manager or exclude the creator), enforcing segregation of duties in the routing logic.

Q88. The organization structure changes often. How do you keep routing resilient to reorg changes?

Explanation: Route based on organizational roles/reporting relationships and work queues rather than hard-coded operator IDs, so routing follows the org model automatically.

Q89. Business wants certain assignments visible to two teams at once until one takes it. How?

Explanation: Route to a shared work queue accessible by both teams so either can pick it up, with the first to accept taking ownership.

Q90. A manager wants to see and rebalance the team's pending work. Which portal capability supports this?

Explanation: The Case Manager portal provides work queue/worklist visibility and bulk reassignment so managers can monitor and rebalance workloads.

7. SLAs, Goals, Deadlines & Escalation

Q91. The business promises customers a response within 24 hours. How do you enforce this on an assignment?

Explanation: Attach a Service Level (SLA) to the assignment with a goal and a deadline; configure escalation actions that fire when the times are reached.

Q92. You need urgency to rise as an assignment ages. How do you configure urgency changes?

Explanation: Set the SLA's initial urgency and configure urgency increases at the goal, deadline, and passed-deadline intervals so aging work rises in priority.

Q93. Explain the difference between a goal and a deadline on an SLA in a scenario where a task should ideally finish in 4 hours but must finish in 8.

Explanation: Set the goal to 4 hours (target) and the deadline to 8 hours (the commitment); missing the goal is a soft warning, while missing the deadline triggers stronger escalation.

Q94. After the deadline passes, work still isn't done and should keep escalating. How do you configure ongoing escalation?

Explanation: Configure a passed-deadline interval on the SLA that repeatedly raises urgency and runs escalation actions (notify/reassign) until the work completes.

Q95. An SLA should apply to the entire case's resolution time, not a single assignment. How do you set this up?

Explanation: Apply a case-level SLA (on the stage/case) with its own goal/deadline governing overall resolution, separate from assignment-level SLAs.

Q96. On deadline breach, the case must be reassigned to a backup queue and a manager emailed. How do you achieve both?

Explanation: Configure the SLA's escalation actions to reassign the assignment to the backup work queue and trigger a notification/correspondence to the manager.

Q97. Business wants SLA timers to exclude weekends and holidays. How do you handle business calendars?

Explanation: Configure the SLA to use a business calendar so goal/deadline calculations count only working days/hours, skipping weekends and holidays.

Q98. Different customer tiers need different response times. How do you vary SLA times by tier?

Explanation: Parameterize or circumstance the SLA (or drive the interval by a decision) so premium tiers get tighter goals/deadlines than standard tiers.

Q99. A stakeholder asks how urgency affects which work users see first. How do you explain it?

Explanation: Higher urgency assignments sort to the top of worklists/queues and are picked first by get-next-work, so SLA-driven urgency directly influences prioritization.

Q100. You need an SLA to start only when the case reaches a certain stage, not at creation. How?

Explanation: Attach the SLA at that stage/step so its timer begins when work enters that point, rather than at case creation.

Q101. The team wants to reuse one SLA definition across many assignments. What do you do?

Explanation: Create a reusable Service Level rule and reference it from each assignment, so timing/escalation logic is centrally maintained.

Q102. Management wants a report of how often deadlines are missed. How does SLA design support this?

Explanation: SLA outcomes and case timing data can be exposed/reported (via Insights/reports), so missed-deadline metrics are trackable for continuous improvement.

Q103. A one-off urgent case must jump ahead of everything. How do you handle a manual priority bump?

Explanation: Manually raise the assignment's urgency (or via an action) so it outranks others in the queue, overriding the normal SLA-driven order temporarily.

Q104. An SLA escalation should only reassign after two reminder notifications. How do you stage this?

Explanation: Configure multiple intervals: earlier intervals send reminder notifications; a later passed-deadline interval performs the reassignment, staging the escalation.

Q105. The business changes response commitments frequently. How do you make SLA times easy to change?

Explanation: Drive SLA intervals from application settings/decision rules rather than hard-coding, so business users can adjust commitments without deep changes.

Q106. You must ensure a child case's delay escalates the parent. How do you connect them?

Explanation: Use case dependencies/SLAs so that a delayed child raises urgency or triggers escalation actions that notify or affect the parent case.

8. Automation Shapes & Workflow Automation

Q107. A step must calculate a fee automatically before a human reviews the case. Which automation do you use?

Explanation: Use an automation/utility step (or a data transform/decision) that runs the calculation automatically before the assignment, requiring no user action.

Q108. The workflow needs to send a confirmation email automatically at a certain point. How?

Explanation: Add a 'Send email' automation step using a correspondence template populated from case data, so the email sends automatically during processing.

Q109. You must create a related case automatically when the main case reaches a stage. Which shape?

Explanation: Use a 'Create case' automation (child case creation) so the related case is spun off automatically at that point in the lifecycle.

Q110. A step should call an external REST service and store the response. How do you model it?

Explanation: Use an automation/integration step that invokes the connector (or references a data page sourced by it) and maps the response into case data.

Q111. The process must branch based on a computed value with several outcomes. Which shape?

Explanation: Use a decision automation (backed by a decision table/tree or when rule) that routes the flow down the connector matching the outcome.

Q112. You need to update several case fields at once automatically. Which rule/shape is best practice?

Explanation: Use a data transform in an automation step to set/copy multiple fields declaratively, which is preferred over an activity for data manipulation.

Q113. A step should run only if a prior condition is true, otherwise skip. How do you configure it?

Explanation: Add a start-when/skip-when condition on the step using a when rule, so it executes only when the condition holds.

Q114. Business wants a fully automated straight-through case when data is valid. How do you design it?

Explanation: Chain automation steps (validate, decide, calculate, integrate, resolve) with no assignment shapes, so eligible cases complete end-to-end without human involvement.

Q115. A step needs to persist case data to a database automatically. How do you handle it?

Explanation: Use a save/persist automation (or savable data page save plan) to write the data to the system of record at that point in the flow.

Q116. You must attach a generated document to the case automatically. Which automation supports this?

Explanation: Use an attach-content/generate-document automation step so the file is produced and linked to the case without manual attachment.

Q117. A process must wait for an external approval callback before proceeding. Which construct do you use?

Explanation: Use a Wait step (or an assignment awaiting a service callback) that pauses the case until the external event/condition resumes it.

Q118. Multiple automated steps must run one after another reliably. How do you sequence them?

Explanation: Place the automation steps in order within the process; each runs to completion before the next, giving a deterministic automated sequence.

Q119. The client wants to reuse the same set of automated steps in several case types. How?

Explanation: Encapsulate them in a reusable process/flow (or subprocess) and reference it from each case type to avoid duplication.

Q120. An automation step occasionally fails due to a downstream outage. How do you design for failure?

Explanation: Add error handling: route failures to an exception path/alternate stage, queue for retry, and notify support, so the case doesn't silently stall.

Q121. Business wants to run a background job to process queued items periodically. How do you approach it?

Explanation: Use a queued/background processing approach (queue processor/job) so items are handled asynchronously on a schedule, separate from user interaction.

Q122. A CSA is tempted to write an activity for a simple data copy. What is the recommended alternative?

Explanation: Prefer a data transform (or Declare Expression) for the copy; reserve activities for logic that configuration genuinely cannot achieve, to protect guardrails.

9. Decisions: Decision Tables, Trees & When Rules

Q123. Shipping cost depends on weight and destination zone, evaluated as a matrix of ranges. Which decision rule fits?

Explanation: Use a decision table: rows test combinations of weight ranges and zones against the same properties, returning the matching cost — ideal for matrix-style conditions.

Q124. Eligibility follows nested if-then-else logic that branches deeply. Which decision rule fits better?

Explanation: Use a decision tree, which models sequential, nested branching logic more naturally than a flat table.

Q125. You just need a simple true/false test reused in several places (e.g., 'is premium customer'). What do you use?

Explanation: Use a when rule that returns boolean, reusable across visibility, routing, and flow decisions.

Q126. A discount rate is looked up from one input (loyalty tier). Which rule is simplest?

Explanation: Use a map value (or a small decision table) that maps the loyalty tier to a discount rate in a lookup style.

Q127. Business changes pricing thresholds monthly and wants easy edits. How do you make decision logic business-friendly?

Explanation: Author the logic in a decision table (editable in App Studio) so business users can adjust rows/thresholds without developer changes.

Q128. A field's value must be calculated using a decision table directly in App Studio. How do you wire it up?

Explanation: Configure the field as calculated using a decision table, so the field's value is derived from the table based on other case fields.

Q129. The routing of a case should depend on three combined conditions. Which decision rule and why?

Explanation: A decision table, because it evaluates multiple conditions across columns cleanly and returns a single routing result, keeping logic transparent.

Q130. You need the decision to fall back to a default when no row matches. How do you handle it?

Explanation: Configure an 'otherwise'/default return in the decision table or tree so unmatched cases get a defined default outcome instead of failing.

Q131. A when rule is being duplicated across many rules. What is the best practice?

Explanation: Define the when rule once and reference it everywhere (and mark it a relevant record) to promote reuse and single-point maintenance.

Q132. Business logic must consider ranges (e.g., 0-1000, 1001-5000). How do you express ranges in a decision table?

Explanation: Use comparison operators (>, <=, between) in the table columns so each row matches a numeric range, returning the corresponding result.

Q133. A decision result must be reused by both UI visibility and flow routing. How do you avoid duplicating logic?

Explanation: Encapsulate the logic in a single when/decision rule and reference it from both the visibility condition and the flow decision.

Q134. You must audit how a decision outcome was reached during testing. What tool helps?

Explanation: Use the rule's run/preview and the Tracer to see which decision-table row or tree branch evaluated true and why.

Q135. The team wants to test a decision table's logic in isolation. How do you validate it?

Explanation: Run the decision table with sample inputs (unit test/preview) to confirm each combination returns the expected result before wiring it into flows.

Q136. A calculation depends on values that change; recompute must be automatic. Which rule type crosses into declarative territory?

Explanation: Use a Declare Expression for automatic recomputation; use decision tables/trees for conditional selection of a value based on inputs.

Q137. Business wants both a computed price and a reason code from the same logic. How do you return multiple results?

Explanation: Have the decision table return multiple result columns (price and reason), or set multiple properties, so one evaluation yields both outputs.

Q138. A stakeholder asks when NOT to use a decision table. What do you advise?

Explanation: Avoid decision tables for deeply nested sequential logic (use a tree) or for simple boolean checks (use a when rule); match the rule type to the logic shape.

10. Child Cases, Duplicates, Wait & Dependencies

Q139. An order must trigger separate fulfillment and billing sub-processes that progress independently. How do you model this?

Explanation: Spin off child cases for fulfillment and billing so each has its own lifecycle and can progress independently of the parent order.

Q140. The parent case must not close until all its child cases are resolved. How do you enforce this?

Explanation: Use a Wait step or case dependency on the parent that waits for all children to reach a resolved status before continuing to resolution.

Q141. The parent should proceed as soon as any one child completes. How do you configure the dependency?

Explanation: Configure the parent's wait/dependency to resume when any (rather than all) child cases meet the completion condition.

Q142. Child cases need some data from the parent at creation. How do you pass it down?

Explanation: Map/propagate parent data to the child at instantiation (via the create-case data mapping/data transform), so children start with the needed context.

Q143. Customers sometimes submit the same request twice. How do you detect and handle duplicates?

Explanation: Configure duplicate case detection with weighted matching conditions (e.g., same customer + same item within a window) and let users resolve or merge flagged duplicates.

Q144. You need to define what makes two cases 'the same' for duplicate detection. How do you configure the criteria?

Explanation: Define must-match and weighted-match conditions in the duplicate search so cases are flagged as potential duplicates based on those fields and thresholds.

Q145. A case must pause until a specific date before continuing. How do you implement the delay?

Explanation: Use a Wait step configured to pause until a specified date/time, then automatically resume processing.

Q146. A case must pause until a condition on its data becomes true. Which construct fits?

Explanation: Use a Wait step with a case-dependency/condition type that resumes when the specified condition is satisfied.

Q147. Business wants child cases visible and manageable from the parent. How do you support this?

Explanation: The case hierarchy links parent and children so users can navigate from the parent to its child cases and monitor their status.

Q148. A child case fails; the parent should take an alternate path. How do you handle child failure?

Explanation: Have the child set a status/outcome the parent's dependency evaluates, routing the parent to an exception/alternate path when a child fails.

Q149. You want to avoid creating a child case if one already exists for the same item. How?

Explanation: Add a condition (or duplicate check) before the create-case step so the child is only created when no matching one exists.

Q150. Several child cases of the same type must be created from a list of items. How do you create them in bulk?

Explanation: Iterate the item list to create one child case per item (create-case for each entry), passing each item's data to its child.

Q151. The parent needs to aggregate results from all children after they finish. How do you collect them?

Explanation: After the wait-for-all completes, read child case data (via a report/data page over children) to aggregate results into the parent.

Q152. A dependency risks deadlock if a child waits on the parent and vice versa. How do you avoid it?

Explanation: Design one-directional dependencies (parent waits on children), avoid circular waits, and use clear completion conditions so cases can always progress.

Q153. Duplicate detection is flagging too many false positives. How do you tune it?

Explanation: Adjust the weighted match conditions and threshold so only genuinely similar cases are flagged, reducing false positives.

Q154. A long-running case must wait for a human milestone in another department. How do you coordinate?

Explanation: Model the other department's work as a child case (or dependency) and have the parent Wait until that child/milestone resolves before resuming.

11. Data Pages, Sourcing & System of Record

Q155. A country list rarely changes and is used everywhere. Which data page scope gives the best performance?

Explanation: Use a node-level, read-only data page so the list loads once per node and is shared by all users, minimizing repeated loads.

Q156. User-specific preferences must not leak between sessions. Which data page scope do you choose?

Explanation: Use a requestor-level data page so the data is isolated to that user's session and never shared across users.

Q157. Data tied to the currently open case should not persist beyond it. Which scope fits?

Explanation: Use a thread-level data page so the data lives with the open case/thread and is cleared when that context ends.

Q158. Customer details must be fetched fresh from a CRM each time a specific customer is opened. How do you design the data page?

Explanation: Use a read-only data page keyed by customer ID, sourced from a connector to the CRM, so each key loads that customer's current data on demand.

Q159. You need to load different instances of the same data page for different keys. What feature supports this?

Explanation: Use a parameterized/keyed data page (e.g., D_Customer[CustID]) so one definition caches multiple keyed instances.

Q160. The application must both read and update reference data through a data page. Which data page type do you use?

Explanation: Use a savable (editable) data page with a save plan so the data can be modified and persisted back to the system of record.

Q161. A stakeholder asks why data pages instead of storing everything in the case. How do you justify it?

Explanation: Data pages decouple the app from integration/source details, cache for performance, avoid duplicating source-of-truth data in every case, and centralize data access.

Q162. A data page's cached data is going stale during long sessions. How do you control freshness?

Explanation: Configure the data page's reload/refresh strategy (reload on condition, expire after time) so it refreshes when the data is likely stale.

Q163. Multiple users hit the same expensive reference lookup constantly. How do you reduce load on the source?

Explanation: Cache it in a node-level read-only data page so it is fetched once per node and reused, drastically cutting source calls.

Q164. You need to persist updated customer data back to the CRM. How does a savable data page do this?

Explanation: Configure the savable data page's save plan to call the connector/save logic that writes the data to the CRM system of record on save.

Q165. A data page must be loaded with parameters passed from the case. How do you supply them?

Explanation: Define parameters on the data page and pass values when referencing it, so it loads context-specific data based on those parameters.

Q166. The team asks how to keep integration details out of UI and process rules. What is the pattern?

Explanation: Reference data through data pages/data objects; the data page owns the sourcing/integration, so UI and processes stay decoupled and reusable.

Q167. A data page should load only when actually referenced, not eagerly. How does Pega handle this?

Explanation: Data pages load on demand — their contents populate only when first referenced — which is inherent to how declarative data pages work.

Q168. Reference data differs per environment (dev/test/prod). How do you keep the data page configurable?

Explanation: Drive the data page's source/endpoint from application settings/dynamic system settings so it points to the right environment without rule changes.

Q169. A single case needs data from three systems combined. How do you present it as one clean structure?

Explanation: Use data pages to source each system and combine them into a unified data object/structure the case references, hiding the multi-source complexity.

Q170. You must clear or refresh a data page after a specific business event. How?

Explanation: Configure a reload-on-condition (or programmatically refresh) tied to the event so the data page reloads current data when the event occurs.

12. Data Transforms & Validation

Q171. When a case is created, several fields must be initialized with defaults. Which rule do you use and where?

Explanation: Use a data transform (e.g., pyDefault) on the case so fields initialize with default values automatically at creation.

Q172. You must copy data from a customer data page into the case's fields. What is the best-practice rule?

Explanation: Use a data transform to map/copy the source properties to the target case properties — the preferred, guardrail-friendly approach over an activity.

Q173. Validation must ensure the end date is after the start date, with a clear message. Which rule fits?

Explanation: Use a validate rule comparing the two dates and adding a field message when the end date is not after the start date.

Q174. A field must always be a valid email format. How do you enforce format at the field level?

Explanation: Apply an edit-validate rule (or field type/format) to check the email pattern, giving a format-level validation independent of business logic.

Q175. Validation should only run for premium customers. How do you make validation conditional?

Explanation: Guard the validate rule/step with a when condition (premium customer) so validation runs only in that scenario.

Q176. You need to transform incoming integration data into the case's structure. Which rule and why?

Explanation: Use a data transform to map source fields to target properties cleanly and declaratively, which is easier to maintain than procedural mapping.

Q177. Business wants an error shown next to the offending field, not a generic message. How do you achieve this?

Explanation: Add the validation message to the specific property in the validate rule so the UI displays it inline next to that field.

Q178. A data transform must build a list of line items from source data. How do you handle list mapping?

Explanation: In the data transform, append/loop to populate the target page list from the source, mapping each entry's fields.

Q179. The same default-setting logic is needed in several case types. How do you reuse it?

Explanation: Create a reusable data transform and call it (via 'apply data transform') from each case type's initialization, avoiding duplication.

Q180. A field must be validated against a database (e.g., unique ID). How do you approach server-side validation?

Explanation: Use a validate rule/step that checks against the data (e.g., via a data page/report) server-side and returns a message if the value is invalid.

Q181. You must set a target property only when a source has a value. How do you make transform steps conditional?

Explanation: Add a when condition on the data transform step so it executes only when the source has the required value.

Q182. Client-side required checks pass but bad data still reaches the server. Why, and how do you protect the app?

Explanation: Client-side checks can be bypassed; always back them with server-side validation (validate rules) so data integrity is guaranteed regardless of the client.

Q183. You need to convert/format data (e.g., uppercase a code) during mapping. Where do you do it?

Explanation: Apply the transformation in the data transform step (using functions/expressions) so the value is formatted as it is mapped.

Q184. A validation must compare a field against a value returned by a decision table. How do you combine them?

Explanation: Reference the decision table's result in the validate rule's condition so validation reflects the business logic captured in the table.

Q185. The team wants initialization logic to be traceable and maintainable. Why prefer data transforms over activities?

Explanation: Data transforms are declarative, easier to read/maintain, and score better on guardrails than procedural activities for data initialization/mapping.

Q186. A calculated total must be validated to not exceed a limit. How do you sequence calculation and validation?

Explanation: Compute the total (Declare Expression/data transform), then run a validate rule/constraint that checks it against the limit and messages on breach.

13. Declarative Rules

Q187. A 'Total' field must always equal the sum of line items without anyone clicking calculate. Which declarative rule?

Explanation: Use a Declare Expression with forward chaining so Total recomputes automatically whenever any contributing value changes.

Q188. When a customer's status changes, an activity must run automatically. Which declarative rule?

Explanation: Use a Declare OnChange rule that triggers the activity automatically when the watched property changes value.

Q189. A value should be computed only when it is actually read, not eagerly. Which chaining approach?

Explanation: Use backward chaining (compute on demand) so the value is derived only when needed, rather than recalculating proactively.

Q190. You must enforce that discount never exceeds 50% continuously. Which declarative rule fits?

Explanation: Use a Declare Constraint that continuously validates the discount and raises a message whenever it would exceed 50%.

Q191. Data saved to the database must trigger follow-up processing automatically. Which rule?

Explanation: Use a Declare Trigger that runs an activity when instances of the class are saved/updated/deleted.

Q192. Embedded list data must be reportable. Which declarative rule exposes it?

Explanation: Use a Declare Index to project the embedded/list data into an index table so it can be searched and reported on.

Q193. A stakeholder asks the difference between forward and backward chaining with an example. How do you answer?

Explanation: Forward chaining recalculates a target when a source changes (e.g., Total updates when price changes); backward chaining computes a value on demand when read (e.g., a rarely used derived figure).

Q194. A calculation depends on another calculated value. How do declarative rules handle the dependency network?

Explanation: Pega maintains a dependency network so changing a base value cascades recalculation through dependent Declare Expressions automatically in the right order.

Q195. You need a derived field but worry about performance of constant recalculation. How do you decide chaining direction?

Explanation: If the value is read rarely, use backward chaining (on demand); if it is displayed/used often and inputs change occasionally, forward chaining keeps it always current.

Q196. Business logic must run automatically on change but only under a condition. How do you scope a Declare OnChange?

Explanation: Guard the OnChange with a condition so the triggered activity runs only when the change meets the specified criteria.

Q197. A CSA is tempted to recalculate a total inside an activity on submit. Why prefer a declarative rule?

Explanation: A Declare Expression keeps the value correct everywhere automatically without relying on a specific activity firing, improving reliability and guardrails.

Q198. You must validate a relationship between two properties continuously, not just on submit. Which rule?

Explanation: Use a Declare Constraint, which enforces the relationship declaratively whenever the involved properties change, not only at submit time.

Q199. A derived property is not updating as expected. How do you debug declarative processing?

Explanation: Use the Tracer (with declarative events) and the Clipboard to see when the dependency network fires and whether inputs are changing as expected.

Q200. The team wants event-driven side effects when records change in the DB. Which declarative rule is appropriate?

Explanation: A Declare Trigger, which responds to database save/update/delete events with an activity, is the right event-driven declarative mechanism.

Q201. You need search/reporting on data inside a page list that lives in the BLOB. What must you configure?

Explanation: Configure a Declare Index so the page-list data is exposed to an index table, making it queryable for search and reports.

Q202. A stakeholder asks why declarative rules are 'better' than always coding logic. How do you summarize?

Explanation: They keep relationships automatically consistent without explicit calls, reduce the chance of stale data, are visible/maintainable, and align with best-practice guardrails.

14. Reuse, Relevant Records & Modular Architecture

Q203. Several applications in the enterprise need the same 'Address validation' logic. Where do you place it?

Explanation: Place shared logic in a higher reuse layer (framework/organization per the Enterprise Class Structure) so multiple implementation-layer apps inherit and reuse it.

Q204. Developers keep rebuilding the same fields and views. How do you make the good ones easy to find and reuse?

Explanation: Mark them as relevant records so they surface as recommended reusable components when other developers build, promoting consistent reuse.

Q205. A rule must behave differently for the US vs the EU without duplicating the whole rule. How do you handle it?

Explanation: Circumstance the rule on region so a base version handles the default and a circumstanced variant applies for the specific region.

Q206. You need a temporary parallel development effort merged back later. Which construct do you use?

Explanation: Use a branch (branch ruleset) for the parallel work, then merge it back into the base ruleset when complete.

Q207. A specialized behavior is needed only for one case class lower in the hierarchy. How do you specialize?

Explanation: Define the specialized rule on the more specific class so rule resolution selects it for that class while others use the general rule.

Q208. The client wants to override a framework rule in their implementation layer. How does this work?

Explanation: Create the rule in a higher ruleset/implementation layer so rule resolution picks the override ahead of the framework version in the stack.

Q209. Two teams work on the same app simultaneously. How do you prevent them from overwriting each other?

Explanation: Have each team develop in separate branches, then merge branches into the base with conflict resolution, keeping work isolated until integration.

Q210. Business wants shared data objects (Customer, Product) reused across many case types. How do you organize them?

Explanation: Define them once in a shared reuse layer and reference them from all case types, so the data model is centralized and consistent.

Q211. A rule should apply only after a certain go-live date. How do you time-box it?

Explanation: Use date-based circumstancing so the rule variant becomes effective on/after the specified date automatically.

Q212. How do you decide what belongs in the organization layer vs the implementation layer?

Explanation: Enterprise-wide, broadly reusable assets go in the organization layer; app-specific specializations go in the implementation layer, following the ECS.

Q213. A common process is copy-pasted into five case types. What is the maintainable alternative?

Explanation: Extract it into a reusable process/subprocess (or case type) and reference it, so one definition serves all five and updates propagate.

Q214. You need to package a completed feature to move to another environment. What do you use?

Explanation: Create a product (RAP) rule listing the rules/data to include, then export/import it (or use Deployment Manager) to migrate the feature.

Q215. A locked production ruleset must receive an urgent change. What is the correct process?

Explanation: Make the change in an appropriate (unlocked) development ruleset/branch, test, then promote a new locked version through the release process — not by editing production directly.

Q216. Stakeholders ask how circumstancing avoids rule sprawl. How do you explain it?

Explanation: Instead of many near-duplicate rules, one base rule plus conditional variants (by property/date/template) keeps variations organized and maintainable.

Q217. A reusable component must be discoverable by business users in App Studio. How do you promote it?

Explanation: Mark it as a relevant record and give it clear naming so it appears as a suggested, reusable option in the authoring experience.

Q218. The app must support many divisions with slight differences. How do you architect for reuse plus specialization?

Explanation: Build shared behavior high in the class structure and specialize per division via lower classes/circumstancing, maximizing reuse while allowing differences.

15. Security & Access (RBAC, ABAC, Personas)

Q219. Only supervisors may perform a 'Waive Fee' action. How do you restrict it?

Explanation: Create a privilege for the action, gate the flow action/step with it, and grant the privilege only to the supervisor role via that role's access settings.

Q220. Agents must see only cases belonging to their own region. How do you enforce row-level security?

Explanation: Use attribute-based access control (an access control policy) comparing the operator's region to the case's region to permit or deny access dynamically.

Q221. You need to map users to what they can do based on their job. Which model and Pega feature?

Explanation: Use role-based access control via access groups and roles/AROs, and configure it through the Persona access landing page / Access Manager.

Q222. A persona should be blocked from developing a specific case type. How do you configure this?

Explanation: Restrict the persona's access to that case type (deny the relevant actions) through the Persona access/Access Manager configuration.

Q223. Sensitive SSN data must be hidden from most users and encrypted at rest. What do you configure?

Explanation: Mask/hide it in the UI for unauthorized roles, apply access control policies, and enable property-level encryption so it is protected in storage.

Q224. The company uses corporate SSO. How do you configure Pega authentication?

Explanation: Configure an authentication service using SSO (e.g., SAML 2.0 or OpenID Connect) so login is delegated to the corporate identity provider.

Q225. Explain the difference between authentication and authorization using a login scenario.

Explanation: Authentication verifies the user's identity at login (who they are); authorization then determines what that verified user is allowed to see and do.

Q226. A role must be able to read but not update certain cases. How do you configure the access level?

Explanation: Configure the role's access to that class/case to grant read but deny update (via AROs/access control), enforcing least privilege.

Q227. An access rule must explicitly forbid an action even if another grant exists. Which rule takes precedence?

Explanation: An Access Deny explicitly forbids access and overrides grants, so it is used to hard-block specific actions for a role.

Q228. Business wants field-level differences: some users edit a field, others only view it. How?

Explanation: Combine role/persona-based visibility with access control so the field is editable for authorized roles and read-only/hidden for others.

Q229. A CSA associates an operator with an app, portal, and roles. Which rule holds this context?

Explanation: The access group ties the operator to the application, available portals, and roles — it is the operator's security context.

Q230. You must ensure only authenticated external systems can call a Pega service. How do you secure inbound access?

Explanation: Configure the service package with authentication (e.g., OAuth/credentials) so only authorized external callers can invoke the service.

Q231. Security policy requires enforcing password/lockout rules. Where do you configure platform security policies?

Explanation: Configure the platform's security policies (authentication/lockout settings) to enforce password strength, lockout, and related protections.

Q232. A reorganization changes who should access what. How do you make access resilient to org changes?

Explanation: Base access on roles/personas mapped to job function rather than individual operators, so access follows the role assignments as people move.

Q233. You need dynamic access that depends on both user and case attributes. Which model fits?

Explanation: Use attribute-based access control (access control policies) that evaluate user and object attributes at runtime for fine-grained, dynamic access.

Q234. A stakeholder asks how personas simplify security setup. How do you explain it?

Explanation: Personas group users by how they interact with the app and map to roles/access, so you configure access per persona once rather than per individual user.

16. Reporting with Insights

Q235. Management wants a table of open cases by status they can explore interactively. Which Pega '25 feature do you use?

Explanation: Use Insights — Pega's reporting experience — to build an interactive table/chart-based report of open cases grouped by status.

Q236. A stakeholder wants a visual chart of cases created per month. How do you build it?

Explanation: Create a chart-based Insight aggregating case counts by creation month, so it renders as a visual trend for stakeholders.

Q237. Users want to filter a report to only their region's cases. How do you add filtering?

Explanation: Add filter conditions to the Insight/report (e.g., region equals the user's region) so results are limited accordingly, optionally parameterized.

Q238. A report must aggregate totals and subtotals by category. Which report style fits?

Explanation: Use a summary-style report/Insight that groups by category and aggregates (count/sum/avg) with subtotals, rather than a flat list.

Q239. A field used for filtering makes the report slow. What design change helps?

Explanation: Expose/optimize that property into a database column so the report queries a real column instead of the BLOB, improving performance.

Q240. Business wants to report on data inside an embedded list. What must be configured first?

Explanation: Configure a Declare Index to expose the embedded/list data to an index table so the report/Insight can query it.

Q241. Users need to build their own reports without developer help. How does Insights support this?

Explanation: Insights provides a self-service authoring experience so business users can create and adjust reports/charts on their own.

Q242. A report must combine data from associated classes (case plus customer). How do you do it?

Explanation: Join the associated class in the report definition/Insight so fields from both the case and the related customer appear together.

Q243. Leadership wants the report emailed weekly. How do you distribute it?

Explanation: Schedule the report/Insight for recurring distribution so output is delivered to recipients automatically on the defined cadence.

Q244. A dashboard should show several key metrics at a glance. How do you assemble it?

Explanation: Combine multiple Insights/widgets on a dashboard/landing page so the key metrics display together for quick monitoring.

Q245. The team needs both detailed rows and a visual summary of the same data. How do you present both?

Explanation: Provide a list Insight for detail and a chart/summary Insight for the aggregate, giving users both perspectives on the same dataset.

Q246. A report should only include cases resolved in the last 30 days. How do you express the date filter?

Explanation: Add a filter condition on the resolved date relative to today (last 30 days), so only recent resolutions are included.

Q247. Business wants to compare this quarter vs last quarter. How do you structure the report?

Explanation: Group/aggregate by quarter (and optionally use comparison columns) so the Insight shows the two periods side by side.

Q248. A metric must count only high-priority cases. How do you constrain the aggregation?

Explanation: Add a filter for high priority before aggregating so the count reflects only qualifying cases.

Q249. Reports keep breaking when property names change. How do you build them more robustly?

Explanation: Base reports on stable exposed/optimized properties and shared field definitions so schema changes are managed centrally and consistently.

Q250. A stakeholder asks how Insights differs from just exporting raw data. How do you frame the value?

Explanation: Insights provides interactive, filterable, visual, self-service analysis inside the app with governed data, rather than static one-off exports.

17. Integration (Connectors, Services & REST)

Q251. Pega must retrieve live pricing from an external REST API. Which rule and pattern do you use?

Explanation: Create a Connect-REST rule to the endpoint and invoke it from a read-only data page so the app references clean, decoupled pricing data on demand.

Q252. An external system must create cases in Pega. What do you configure?

Explanation: Configure an inbound service (e.g., Service REST) in a service package with authentication, mapping the incoming payload to case creation logic.

Q253. Distinguish a connector and a service using a scenario with a partner API and an inbound webhook.

Explanation: Calling the partner API outbound uses a connector; the inbound webhook that lets the partner call into Pega uses a service.

Q254. Outbound API credentials expire and must be managed securely. How do you handle authentication?

Explanation: Store credentials/tokens in an authentication profile (e.g., OAuth 2.0) referenced by the connector, keeping secrets in a keystore rather than in the rule.

Q255. You must map a nested JSON response into Pega properties. How do you approach mapping?

Explanation: Configure the connector's response data mapping (or a data transform) to map the JSON structure into the corresponding clipboard properties/data pages.

Q256. A legacy system only exposes SOAP. How do you integrate with it?

Explanation: Use a Connect-SOAP rule based on the WSDL to call the SOAP operations, mapping requests/responses to Pega data.

Q257. Integration must not couple the UI/process to the external system's details. What pattern enforces this?

Explanation: Front the integration with data pages/data objects (a data/integration layer) so UI and process rules depend only on the abstraction, not the endpoint.

Q258. An external call is slow and sometimes times out, stalling the case. How do you design for resilience?

Explanation: Add timeouts/retries, handle errors by routing to an exception path or queuing for retry, and decouple via a data page so failures don't hang the case.

Q259. You need to process a batch file dropped by another system. Which integration type fits?

Explanation: Use a file listener/file service to pick up and process the incoming file on a schedule or on arrival.

Q260. Different environments have different API endpoints. How do you avoid hard-coding URLs?

Explanation: Drive endpoints from dynamic system settings/application settings so the connector points to the correct environment without rule changes.

Q261. You must expose a data page's content to an external caller. How do you serve it?

Explanation: Create an inbound service that returns the data (sourced from the data page) with appropriate authentication and mapping to the response payload.

Q262. A REST call needs custom headers and query parameters at runtime. How do you configure them?

Explanation: Configure the Connect-REST rule's headers/query parameters (parameterized), supplying values at invocation so the request is built dynamically.

Q263. Business wants asynchronous processing of an integration so users aren't blocked. How?

Explanation: Queue the integration work (queue processor/background job) so it runs asynchronously and the user isn't held up waiting for the external system.

Q264. You need to test an integration without the real external system available. How do you approach it?

Explanation: Use simulated/stub data or a test data source for the data page/connector so you can develop and test independently of the live system.

Q265. A partner sends data in XML but Pega needs it as case data. How do you transform it?

Explanation: Map the XML to Pega properties via the service/connector data mapping (and a data transform if needed) to convert it into the case structure.

Q266. Security requires that integration secrets never appear in exported rules. How do you comply?

Explanation: Store secrets in a keystore and reference them from authentication profiles, so exported rules contain references, not the secrets themselves.

18. DevOps, Testing, Debugging & Troubleshooting

Q267. You need confidence that a rule keeps working as the app evolves. What testing do you add?

Explanation: Create unit tests (e.g., PegaUnit test cases) for the rule so regressions are caught automatically as the application changes.

Q268. Management wants a high percentage of tested rules. How do you track and improve coverage?

Explanation: Use the application's test coverage tooling to measure covered rules and add unit tests where coverage is low, improving overall quality.

Q269. A flow produces an incorrect result at runtime. Which tool do you use to see step-by-step execution?

Explanation: Use the Tracer to follow rule execution (flow, data pages, decisions) step by step and pinpoint where the logic diverges.

Q270. You need to inspect the current data/pages during a case. Which tool helps?

Explanation: Use the Clipboard tool to view the in-memory pages and property values for the current thread/requestor.

Q271. A rendered UI element behaves oddly and you must find the underlying rule. Which tool do you use?

Explanation: Use Live UI to map the on-screen element back to its underlying view/section rule and navigate to it quickly.

Q272. You want to promote a feature from Dev to Test reliably and repeatably. What do you set up?

Explanation: Set up a CI/CD pipeline (Pega Deployment Manager) to package and promote rules/data through environments automatically and consistently.

Q273. A production issue can't be reproduced. What is the safe troubleshooting approach?

Explanation: Reproduce it in a lower environment using the same data/config, then use Tracer, Clipboard, and logs to diagnose without risking production.

Q274. Rule resolution is picking an unexpected rule version. How do you investigate?

Explanation: Inspect the ruleset stack and the rule's availability/circumstance/class, and use the Tracer/rule inspection to see which instance resolves and why.

Q275. Performance degrades under load. What do you examine first?

Explanation: Check the diagnostic tools/logs (e.g., PAL/PDC) for expensive operations — heavy activities, un-optimized report queries, or excessive data-page loads — and address them.

Q276. A team wants to catch guardrail violations early. How do you enforce quality gates?

Explanation: Monitor the guardrail compliance score and add pipeline checks/tests so violations and low compliance are flagged before promotion.

Q277. A data page returns no data intermittently. How do you debug it?

Explanation: Trace the data page load, inspect its source (connector/report), verify parameters/keys, and check the Clipboard to see what actually loaded.

Q278. You must verify a decision table returns correct results for many combinations. How do you test efficiently?

Explanation: Write unit tests covering representative combinations (including boundaries and the default) so the table's behavior is verified automatically.

Q279. A change broke another feature after promotion. How do you prevent this in future?

Explanation: Expand automated regression tests and use the pipeline to run them on every promotion, catching breakages before they reach higher environments.

Q280. Logs are noisy and hard to search during debugging. How do you focus your investigation?

Explanation: Use targeted tracing (filtered Tracer), correlate with the specific case/thread, and use the diagnostic tools to isolate the relevant events.

Q281. You want business users to log bugs and stories against the app. Which capability supports this?

Explanation: Use Agile Workbench/Developer Assistant so users can capture user stories, bugs, and feedback directly linked to the application.

Q282. A rule works in Dev but fails in Test. What common causes do you check?

Explanation: Check environment-specific settings (endpoints, credentials), ruleset versions promoted, data/reference differences, and access/security configuration between environments.

19. Localization, Application Settings & Application Variables

Q283. The app must be delivered in Spanish for one region. How do you localize the content?

Explanation: Use Pega's localization support to translate field labels, messages, and UI text (via language packs/translation), so the app renders in Spanish without rebuilding it.

Q284. Users in different time zones see wrong times. How do you handle time zones?

Explanation: Configure locale/time-zone handling so date-times display in each user's local time while stored consistently, using Pega's localization features.

Q285. A configurable threshold (e.g., approval limit) changes per environment and over time. How do you expose it?

Explanation: Store it as an application setting/variable so it can be changed without editing rules and can differ per environment.

Q286. Business wants to toggle a feature on/off without a deployment. How do you design for this?

Explanation: Drive the feature with an application setting/toggle so business can enable/disable it via configuration rather than a code change.

Q287. The same message text is reused in many places and must be translatable. How do you manage it?

Explanation: Define reusable, localizable message/field text so translations apply centrally and consistently everywhere the text is used.

Q288. Currency and number formats differ by locale. How does the app adapt?

Explanation: Rely on locale-aware formatting so numbers, currency, and dates render according to each user's locale automatically.

Q289. A setting must differ between Dev, Test, and Prod. How do you keep one rule but different values?

Explanation: Use environment-specific dynamic system settings/application settings so the same rule reads the correct value per environment.

Q290. You must add a new language after go-live with minimal rework. How does localization design help?

Explanation: Because translatable text is externalized into language packs, adding a language is largely a translation effort rather than a rebuild.

Q291. A hard-coded value keeps needing developer changes. What is the better pattern?

Explanation: Externalize it as an application setting/variable so business or ops can change it via configuration, reducing developer involvement.

Q292. Stakeholders ask how localization affects reports and correspondence. How do you answer?

Explanation: Labels, messages, and correspondence templates can be localized too, so reports and communications also render in the user's language where configured.

Q293. You need consistent branding text (company name) that might change. How do you manage it?

Explanation: Store it as an application setting/variable and reference it, so a single change updates it everywhere it appears.

Q294. A field label must read differently in two languages. How does Pega resolve which to show?

Explanation: The platform selects the label from the appropriate language pack based on the user's locale, so each user sees their language's label.

20. Rule Resolution, Class Structure & Core Foundations

Q295. At runtime two rules with the same name exist in different rulesets. How does Pega pick one?

Explanation: Rule resolution searches the requestor's ruleset stack top-down and applies class hierarchy, availability, circumstance, and date, selecting the single most appropriate rule.

Q296. Explain pattern vs directed inheritance using a class like MyCo-HR-Work-Leave.

Explanation: Pattern inheritance walks up by trimming the dash-separated name (Leave to Work to HR...) automatically; directed inheritance follows the explicitly configured parent class. Pattern is checked first, then directed.

Q297. Several concrete case classes should share one database table and ID prefix. What construct do you use?

Explanation: Group them under a class group (work pool) so they share a table and a common work ID prefix.

Q298. A rule must be reusable across many classes. Where do you define its Applies-To class?

Explanation: Define it on a higher, more abstract class in the hierarchy so rule resolution makes it available to the classes that inherit from it.

Q299. A developer checks out a rule to edit it. What does check-out/check-in do?

Explanation: Check-out creates a private editable copy so others aren't affected; check-in commits the changes back, enabling controlled collaborative editing.

Q300. A locked ruleset version prevents edits. Why lock versions and how do you make changes?

Explanation: Locking enforces controlled change and stable releases; to change, create/edit in an unlocked version or branch, then promote a new version.

Q301. How does the ruleset stack determine which rules a user sees?

Explanation: The user's access group/application defines the rulesets and versions in scope (the stack); rule resolution only considers rules within that stack.

Q302. A rule variant should apply only when a property has a specific value. Which mechanism?

Explanation: Circumstance the rule on that property so the variant resolves when the value matches, and the base rule applies otherwise.

Q303. Explain the difference between a rule that is 'available', 'blocked', and 'withdrawn'.

Explanation: Available rules participate in resolution; blocked rules are prevented from being used; withdrawn rules effectively remove a rule from resolution at that level so a lower one applies.

Q304. A case ID looks like 'A-1023'. Where is that prefix and numbering defined?

Explanation: The prefix/numbering is defined on the class group (work pool) configuration, which controls how new case IDs are generated.

Q305. Why does Pega store rules separately from work/data in the database?

Explanation: Separating the rules schema from the data schema lets rules be versioned/migrated independently of runtime work and reference data, supporting clean deployment.

Q306. A rule change in a higher layer should override a framework rule. How does layering achieve this?

Explanation: Because the implementation ruleset sits above the framework in the stack, rule resolution finds the implementation rule first, overriding the framework version.

Q307. How do you make a rule effective only after a future date?

Explanation: Use date-based circumstancing/availability so the rule variant becomes effective on/after the specified date and resolves accordingly.

Q308. A stakeholder asks what 'specialization' means in Pega. How do you explain it with an example?

Explanation: Specialization tailors a rule for a narrower context — a lower class, a specific ruleset, or a circumstance (e.g., a region-specific pricing rule) — while a general rule covers the default.

Q309. Two applications must share some rules but not others. How does the application/ruleset structure enable this?

Explanation: Shared rules live in common rulesets/layers both apps build on, while app-specific rules stay in each app's own rulesets, so sharing is controlled by the stack composition.

Q310. How does rule resolution handle a case class that inherits from multiple layers?

Explanation: It evaluates pattern inheritance first (by class name), then directed inheritance to the configured parent, searching up through the layers until it finds the applicable rule in scope.